Lập trình mạng · Chương 5

Bảo mật: TLS và secure socket trong Java

TS. Đặng Ngọc Hùng

Chương 5: Các chủ đề nâng cao

1 Bảo mật

- TLS và secure socket trong Java
- Xác thực và phân quyền với OAuth 2.0 và JWT

2 Hiệu năng

- Lời gọi không chặn với CompletableFuture và gRPC async stub
- NIO, Selector và mẫu Reactor

3 Độ tin cậy

- Retry, request ID và checksum trên UDP
- Backoff, circuit breaker và health check

4 IoT

- Bài toán ứng dụng: giám sát cảm biến với MQTT và CoAP

Dữ liệu dạng rõ, server không được xác thực

1. Echo server, web server, dịch vụ REST, gRPC ở các chương trước gửi dữ liệu **dạng rõ**: máy nào trên đường truyền cũng đọc và sửa được
2. Client không biết có đang kết nối tới **đúng server** hay không
3. **TLS**: mã hóa đường truyền và xác thực server bằng **chứng chỉ**

> **Câu hỏi của phần:** TLS bảo vệ kết nối thế nào, và dùng TLS trong Java ra sao?

Trên đường truyền, dữ liệu của echo server hiện nguyên văn

Java

```
Socket server = new Socket("127.0.0.1", 9199);
System.out.println(direction + " " + n + " bytes: " + show(buf, n));
out.write(buf, 0, n);  // chuyển tiếp nguyên vẹn
```

Kết quả chạy

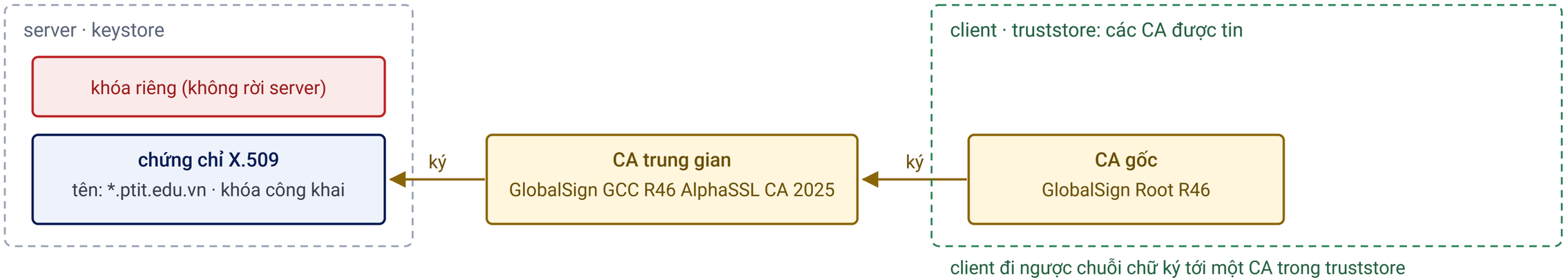
```
tap on port 9200 -> 127.0.0.1:9199
client -> server 7 bytes: "hello\n"
server -> client 7 bytes: "hello\n"
```

Handshake TLS: thỏa thuận khóa, xác thực server, rồi mã hóa



Handshake TLS: trao đổi phần khóa để có khóa phiên, server chứng minh danh tính bằng chứng chỉ; sau đó mọi dữ liệu được mã hóa.

Chứng chỉ, keystore, truststore và kiểm tra hostname



so tên trong chứng chỉ với host đã kết nối:

`www.ptit.edu.vn` ✓ `wrong.host.badssl.com` ✗ thất bại

echo server của phần: chứng chỉ tự ký cho localhost;
chính chứng chỉ đó được đưa vào truststore của client

Keystore (server): khóa riêng và chứng chỉ. **Truststore** (client): các CA được tin. Client kiểm tra chuỗi tin cậy rồi **kiểm tra hostname**.

Tạo keystore và truststore bằng keytool

Bash

```
keytool -genkeypair -alias echo -keyalg EC -validity 365 \
    -dname "CN=localhost" -ext "SAN=dns:localhost" \
    -keystore server.p12 -storepass changeit
keytool -exportcert -alias echo -rfc -file server.crt \
    -keystore server.p12 -storepass changeit
keytool -importcert -alias echo -file server.crt -noprompt \
    -keystore client-trust.p12 -storepass changeit
```

Kết quả chạy

```
$ sh make-certs.sh
Generating 384 bit EC (secp384r1) key pair and self-signed certificate (SHA384withECDSA) with a validity of 365 days
        for: CN=localhost
Certificate stored in file <server.crt>
Certificate was added to keystore
```

Server TLS chỉ đổi cách tạo listening socket; Tap chỉ thấy bản ghi mã hóa

Java

```
System.setProperty("javax.net.ssl.keyStore", "server.p12");  // khóa riêng và chứng chỉ của server
System.setProperty("javax.net.ssl.keyStorePassword", "changeit");
listener = SSLServerSocketFactory.getDefault().createServerSocket(9199);  // listening socket TLS
```

Kết quả chạy

```
tap on port 9200 -> 127.0.0.1:9199
client -> server 410 bytes: 16 03 03 01 95 01 00 01 91 03 03 d6 ec f0 ec 6b ...
server -> client 127 bytes: 16 03 03 00 7a 02 00 00 76 03 03 3d 71 7b 62 82 ...
server -> client 6 bytes: 14 03 03 00 01 01
client -> server 6 bytes: 14 03 03 00 01 01
server -> client 70 bytes: 17 03 03 00 41 9e 4e 92 57 06 be d8 76 36 a6 22 ...
server -> client 457 bytes: 17 03 03 01 c4 c5 1d d2 f2 f4 29 41 11 08 d8 54 ...
server -> client 150 bytes: 17 03 03 00 91 41 9a 88 06 5c f3 0b 70 18 17 b2 ...
server -> client 90 bytes: 17 03 03 00 55 ff 72 1e 70 ef 32 6a 82 71 c2 53 ...
client -> server 90 bytes: 17 03 03 00 55 f1 5e bb 9e 9d f3 e5 84 14 b0 45 ...
server -> client 711 bytes: 17 03 03 02 c2 bb 6f 44 25 90 85 cc 2c 5f bc 4d ...
client -> server 45 bytes: 17 03 03 00 28 da c0 0c ca 12 44 4f 54 92 71 95 ...
server -> client 45 bytes: 17 03 03 00 28 81 59 fb 35 c4 db 01 71 78 b4 e6 ...
client -> server 40 bytes: 17 03 03 00 23 83 d2 e9 c4 bf e1 e0 75 77 af 1b ...
client -> server 40 bytes: 17 03 03 00 23 07 2b ec 22 9b cf c7 5c 00 08 c8 ...
```

SSLSocket không tự kiểm tra hostname; đừng tắt kiểm tra

Java

```java
try (SSLSocket socket = (SSLSocket) SSLSocketFactory.getDefault().createSocket(host, 443)) {
    if (hostnameCheck) {
        SSLParameters params = socket.getSSLParameters();
        params.setEndpointIdentificationAlgorithm("HTTPS");  // bật kiểm tra hostname theo chứng chỉ
        socket.setSSLParameters(params);
    }
    socket.startHandshake();  // kiểm tra chuỗi tin cậy, hạn dùng và hostname (nếu bật)
```

Kết quả chạy

```
$ java netlab.tls.TlsCheck ptit.edu.vn expired.badssl.com wrong.host.badssl.com self-signed.badssl.com untrusted-root.badssl.com
ptit.edu.vn: OK TLSv1.3, chain: *.ptit.edu.vn <- GlobalSign GCC R46 AlphaSSL CA 2025 <- GlobalSign Root R46
expired.badssl.com: FAIL CertificateExpiredException: NotAfter: Mon Apr 13 06:59:59 ICT 2015
wrong.host.badssl.com: FAIL CertificateException: No subject alternative DNS name matching wrong.host.badssl.com found.
self-signed.badssl.com: FAIL SunCertPathBuilderException: unable to find valid certification path to requested target
untrusted-root.badssl.com: FAIL SunCertPathBuilderException: unable to find valid certification path to requested target
$ java netlab.tls.TlsCheck --no-hostname-check wrong.host.badssl.com
wrong.host.badssl.com: OK TLSv1.2, chain: *.badssl.com <- YR1 <- Root YR
```

- SSLSocket **không tự kiểm tra hostname**: phải đặt endpoint identification algorithm
- Tắt kiểm tra chứng chỉ hay hostname để chương trình chạy được là **lỗ hổng**

Câu hỏi 1

Trong ví dụ của phần, file client-trust.p12 là truststore. Truststore chứa gì và bên nào dùng?

A. Khóa riêng của server; server dùng để ký trong handshake.

B. Mật khẩu của client; server dùng để xác thực client.

C. Các chứng chỉ được tin (thường là CA); client dùng để kiểm tra chứng chỉ server gửi tới.

D. Khóa phiên đã thỏa thuận; cả hai bên dùng để mã hóa.

Câu hỏi 1

Trong ví dụ của phần, file client-trust.p12 là truststore. Truststore chứa gì và bên nào dùng?

A. Khóa riêng của server; server dùng để ký trong handshake.

B. Mật khẩu của client; server dùng để xác thực client.

C. Các chứng chỉ được tin (thường là CA); client dùng để kiểm tra chứng chỉ server gửi tới.

D. Khóa phiên đã thỏa thuận; cả hai bên dùng để mã hóa.

Truststore chứa các chứng chỉ, tức khóa công khai, mà client tin. Khóa riêng của server nằm trong keystore ở server và không bao giờ được gửi đi; khóa phiên được tính ra trong handshake, không lưu trong file.

Câu hỏi 2

Kẻ tấn công có một chứng chỉ hợp lệ, do CA mà client tin cấp, cho tên miền của chính họ. Nếu client kiểm tra chuỗi tin cậy nhưng không kiểm tra hostname, điều gì xảy ra khi kẻ đó chặn kết nối của client tới example.com?

A. Client chấp nhận server giả, vì chứng chỉ của kẻ tấn công cũng có chuỗi tin cậy hợp lệ.

B. Handshake thất bại vì chứng chỉ không hợp lệ.

C. Dữ liệu vẫn an toàn vì đã được mã hóa.

D. Kẻ tấn công không thể chặn một kết nối TLS.

Câu hỏi 2

Kẻ tấn công có một chứng chỉ hợp lệ, do CA mà client tin cấp, cho tên miền của chính họ. Nếu client kiểm tra chuỗi tin cậy nhưng không kiểm tra hostname, điều gì xảy ra khi kẻ đó chặn kết nối của client tới example.com?

A. Client chấp nhận server giả, vì chứng chỉ của kẻ tấn công cũng có chuỗi tin cậy hợp lệ.

B. Handshake thất bại vì chứng chỉ không hợp lệ.

C. Dữ liệu vẫn an toàn vì đã được mã hóa.

D. Kẻ tấn công không thể chặn một kết nối TLS.

Chuỗi tin cậy chỉ cho biết chứng chỉ do một CA được tin cấp; kiểm tra hostname mới cho biết chứng chỉ cấp cho đúng server client muốn kết nối. Dữ liệu vẫn được mã hóa, nhưng mã hóa tới chính kẻ tấn công.

Tổng kết

1. **TLS** mã hóa đường truyền; **handshake TLS** thỏa thuận khóa phiên và xác thực server

2. **Chứng chỉ X.509** gắn tên với khóa công khai; server giữ **keystore**, client giữ **truststore**

3. **SSLSocket** cần bật **kiểm tra hostname**; không tắt kiểm tra để chương trình chạy được

Từ khóa cần nhớ TLS · handshake TLS · chứng chỉ X.509 · keystore · truststore · kiểm tra hostname · SSLSocket

Mở rộng

Chủ đề nâng cao

- Mutual TLS: xác thực cả client
- Certificate pinning

Bài tập mở rộng

Tạo CA riêng bằng keytool, ký chứng chỉ cho server, cấu hình client chỉ tin CA đó.

Tiếp theo

Bảo mật: xác thực và phân quyền với OAuth 2.0 và JWT